

Documentação Técnica de Arquitetura

Engenharia Reversa, Modularização e Refatoração do ERP de Saúde Pública

Histórico de Versão: Documentação oficial cobrindo todas as etapas de implementação desde a inicialização do repositório até a consolidação da arquitetura distribuída (Commits `bf1210f` a `a708cf2`).

1. Visão Geral do Projeto

O presente documento detalha a estratégia de engenharia de software aplicada na modernização estrutural do ecossistema de um ERP de Saúde Pública. O objetivo principal foi migrar de um paradigma monolítico rigidamente acoplado a um banco de dados legado para uma **arquitetura distribuída, modular e resiliente** em Django. A infraestrutura de dados utiliza o **Neon DB** (Serverless Postgres) e chaves primárias otimizadas com suporte a identificadores baseados em **UUIDv7**.

Seguindo os conceitos de *Bounded Contexts* do **Domain-Driven Design (DDD)**, a aplicação foi segmentada em sub-aplicativos independentes ("ilhas de domínio"). Esta abordagem garante o isolamento de escopo, facilita a manutenção contínua e estabelece fronteiras claras de governança sobre as regras de negócio de saúde do município.

2. Fase 1: Fundação, Documentação e Infraestrutura

Commits de referência: `bf1210f`, `5829232`, `a6f14df`, `3650363`, `1757198`, `3bfc9a3`

A base do ecossistema foi estabelecida adotando-se padrões modernos de engenharia e versionamento:

- **Inicialização e Escopo:** Criação do repositório base (`bf1210f`) e estruturação da documentação inicial de requisitos do Public Health ERP (`5829232`).
- **Integração Neon DB:** Inicialização do projeto Django integrada ao banco Serverless Postgres via Neon DB, viabilizando operações escaláveis na nuvem.
- **Protocolo ASGI:** Configuração nativa voltada ao suporte assíncrono para operações de alta concorrência.
- **Gerenciamento Estrito de Dependências:** Isolamento do ambiente virtual (`requirements.txt`) e congelamento da versão do `django` (`3bfc9a3`) para mitigar quebras por atualizações upstream.

3. Fase 2: Mapeamento de Domínios e Engenharia Reversa

Commits de referência: `f5243fd` a `d0f2de2`

Para absorver o esquema do banco de dados físico sem interferir na operação legada, realizou-se um processo cirúrgico de engenharia reversa automatizada (via `inspectdb` customizado). Cada módulo do

sistema foi isolado em seu próprio app Django, com tabelas mapeadas utilizando chaves temporais indexadas por **UUIDv7** para otimizar consultas históricas e de auditoria clínica.

Os limites geográficos e lógicos de dados foram divididos em oito domínios centrais:

1. **Pacientes:** Cadastro demográfico, prontuário individual e identificação unificada do cidadão.
2. **Atendimentos:** Núcleo transacional responsável pelo ciclo de vida das consultas e fluxos clínicos.
3. **Profissionais:** Cadastro de operadores do sistema, digitadores, médicos e enfermeiros.
4. **Triagem:** Filas de espera e protocolos de acolhimento inicial das unidades de saúde.
5. **Diagnósticos:** Catálogos internacionais de termos clínicos (tabelas CID e CIAP).
6. **Geografia:** Territorialização, mapeamento de microáreas e cadastro domiciliar de saúde.
7. **Institucional:** Governança organizacional do município (Empresas, Filiais e Organograma).
8. **Faturamento:** Regras de convênios, tabelas do SIGTAP/SUS e faturamento de procedimentos (SIA/SIH).

4. Fase 3: Segurança e Otimização de Performance da API

Commits de referência: `687b021`, `cb6f62a`, `b7faa45`, `f2312d8`

Após a modelagem estrutural do banco, os endpoints de exposição REST foram otimizados visando performance em cenários de alta carga e conformidade com legislações de proteção de dados:

- **Privacidade e LGPD:** Implementação de máscaras nativas de segurança no `AtendimentoSerializer` (`687b021`) para ofuscação de dados sensíveis, como o CPF do paciente, diretamente no nível de serialização do backend.
- **Otimização de Consultas (Resolução do Gargalo N+1):** Desenvolvimento da view de alta performance `AtendimentoListAPIView` (`cb6f62a`). A aplicação explícita do método `select_related` do Django ORM força a execução de JOINS otimizados no banco de dados, reduzindo centenas de queries redundantes a uma única instrução SQL unificada durante a listagem de visitas clínicas recentes.

5. Fase 4: A Grande Refatoração (Acoplamento Fraco e Limpeza de Stubs)

Commits de referência: `0c5ee69` a `a708cf2`

A engenharia reversa bruta gera, por padrão, cópias locais estáticas de tabelas estrangeiras (stubs) em cada arquivo `models.py` onde há um relacionamento. No app atendimentos, isso criava uma severa redundância de código e violava o princípio de **Fonte Única da Verdade (Single Source of Truth)**. A refatoração executada organizou estes stubs sob duas linhas de ação arquitetural:

Estratégia A: Redirecionamento e Resolução Tardia (Lazy Evaluation)

Para os stubs que representavam entidades com governança e apps oficiais já existentes no ecossistema, os modelos redundantes locais foram removidos por completo. As propriedades de chaves estrangeiras (`ForeignKey`) na classe principal `Atendimento` foram alteradas para referências em formato de string (ex: `'diagnosticos.Cid'`). O Django resolve essas dependências em tempo de execução, garantindo acoplamento fraco (*loose coupling*).

Estratégia B: Promoção a Modelos Nativos Gerenciados

Tabelas acessórias que orbitavam especificamente a regra de negócio do ato clínico e não possuíam apps de origem foram **promovidas a modelos locais definitivos** dentro de `atendimentos/models.py`, elevando a coesão do domínio clínico.

Matriz de Destino de Domínios e Dependências Resolvidas

Modelo (Stub Original)	App de Origem / Destino Real	Estratégia Aplicada	Justificativa Arquitetural
Empresa / Filial	institucional	Redirecionamento de FK	Centralização das regras organizacionais da prefeitura.
MotivosCancelamento	triagem	Redirecionamento de FK	Parte integrante do fluxo de recepção e controle de filas.
Usuarios	profissionais	Redirecionamento de FK (3 campos)	Entidade global de controle de operadores e profissionais logados.
ProcedimentoCompetencia	faturamento	Redirecionamento de FK	Vinculado diretamente às regras de competência do SIA/SUS.
AcaoProgramaticaGrupo	atendimentos (Local)	Promoção a Modelo Local	Classificação clínica de programas de saúde (Hiperdia, Pré-natal).
TabelaCbo	profissionais	Redirecionamento de FK (2 campos)	Mapeamento da Classificação Brasileira de Ocupações de Saúde.
Cid	diagnosticos	Redirecionamento de FK (2 campos)	Catálogo global da Classificação Internacional de Doenças.
ClassificacaoAtendimento	atendimentos (Local)	Promoção a Modelo Local	Tipagem nativa da consulta (Urgência, Eletiva, Ambulatorial).
Conduta	atendimentos (Local)	Promoção a Modelo Local	Desfecho clínico imediato do atendimento (Alta, Encaminhamento).
EnderecoDomicilio	geografia	Redirecionamento de FK	Governança do cadastro territorial e habitacional.

Modelo (Stub Original)	App de Origem / Destino Real	Estratégia Aplicada	Justificativa Arquitetural
EnderecoUsuarioCadsus	geografia	Redirecionamento de FK	higienização geográfica pareada com o barramento do CADSUS.
NaturezaProcuraTpAtendimento	atendimentos (Local)	Promoção a Modelo Local	Identificação do motivo de busca do cidadão (Demanda Espontânea).
LeitoQuarto	atendimentos (Local)	Promoção a Modelo Local	Infraestrutura física de observação clínica sob gestão do app.
Convenio	faturamento	Redirecionamento de FK	Regulação de fontes pagadoras e convênios municipais.
ClassificacaoRisco	triagem	Redirecionamento de FK	Vinculado ao protocolo clínico de triagem (Manchester).
TipoProcedimentoAtendimento	faturamento	Redirecionamento de FK	Faturamento de procedimentos ambulatoriais especializados.
Ciap	diagnosticos	Redirecionamento de FK	Classificação Internacional de Atenção Primária (Atenção Básica).
AtividadeGrupo	atendimentos (Local)	Promoção a Modelo Local	Mapeamento de atendimentos coletivos e ações comunitárias.
EstabelecimentoCerest	institucional	Redirecionamento de FK	Cadastro dos Centros de Referência em Saúde do Trabalhador.
Equipe	institucional	Redirecionamento de FK	Mapeamento das equipes de saúde da família (eSF / eMulti).

6. Conclusões e Diretrizes de Manutenção

A consolidação da refatoração removeu centenas de linhas de código redundantes, atingindo os objetivos técnicos de design de software limpo (*Clean Code*), eliminação de stubs órfãos e blindagem contra falhas em cascata na camada de persistência.

Diretrizes Mandatórias para Futuras Manutenções:

1. **Preservação do Banco Legado:** Todos os modelos que herdaram ou se conectam a tabelas físicas legadas gerenciadas externamente devem manter a propriedade `managed = False` e a string exata em `db_table` dentro do seu bloco `Meta`, impedindo migrações destrutivas acidentais.
2. **Isolamento de Acoplamento Cross-App:** Fica estritamente proibido importar classes de modelos de outros apps diretamente no cabeçalho do arquivo para declaração de chaves estrangeiras. Deve-se adotar exclusivamente a sintaxe de string tardia (`'nome_app.NomeModelo'`) para mitigar erros de importação circular.
3. **Enriquecimento de Modelos Promovidos:** Caso haja necessidade de expor propriedades internas das tabelas promovidas localmente (ex: listar o nome por extenso de uma `Conduta`), deve-se executar um `inspectdb` cirúrgico focado exclusivamente na tabela alvo para povoar os campos ausentes no modelo.